Lecture – 5
B-tree

M.G.Noel A.S Fernando (PhD)

Professor

NSBM/Dept. of Information Systems Engineering, UCSC

Contents

- Describe B-Trees and B+ Trees
- Key Differences Between B-Trees and B+ Trees
- Basic Operations of B-Trees and B+ Trees
- Applications of B-Trees and B+ Trees
- Q and **A**

B-tree

A **multi-way search tree** (or multi-way tree) is a tree where each node can have more than two children.

A **B-tree** is a specific kind of multi-way search tree with additional properties that make it especially useful for **disk-based storage systems** like databases and file systems.

Definition of a B-tree

- A B-tree of order m is an m-way tree (i.e., a tree where each node may have up to m children) in which:
  1. the number of keys in each non-leaf node is one less than the number of its children and these keys partition the keys in the children in the fashion of a search tree
  2. all leaves are on the same level
  3. all non-leaf nodes except the root have at least $\lceil m / 2 \rceil$ children
  4. the root is either a leaf node, or it has from two to m children
  5. a leaf node contains no more than $m - 1$ keys
- The number m should always be odd

Motivation

- When data is too large to fit in main memory, then the number of disk accesses becomes important.
- A disk access is unbelievably expensive compared to a typical computer instruction (mechanical limitations).
- One disk access is worth about 200,000 instructions.
- The number of disk accesses will dominate the running time.
- When data is too large to fit in main memory, then the number of disk accesses becomes important.
- A disk access is unbelievably expensive compared to a typical computer instruction (mechanical limitations).
- One disk access is worth about 200,000 instructions.
- The number of disk accesses will dominate the running time.

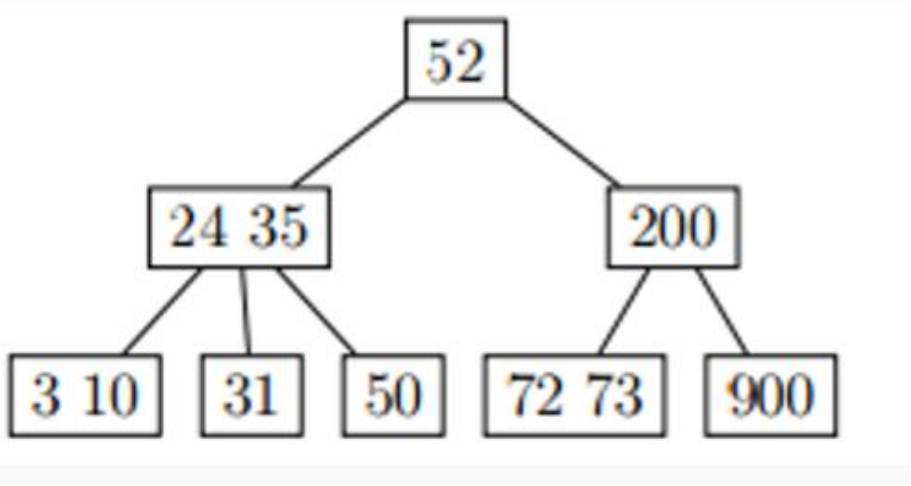
What is a **B-tree**?

- A **B-tree** is a self-balancing search tree data structure that maintains sorted data and allows searches, sequential access, insertions, and deletions in **O(log n)** time.
- It generalizes the binary search tree by allowing each node to have **more than two children**.

- **Properties of a B-tree of order *m***
- **Each node can have at most m children**.
- Each node (except the root) must have at least **⌈m/2⌉ children**.
- Each node can store **up to m-1 keys**.
- The root must have at least 2 children if it is not a leaf.

What is a **B-tree**? Cont..

- Example (B-tree of order m=3):



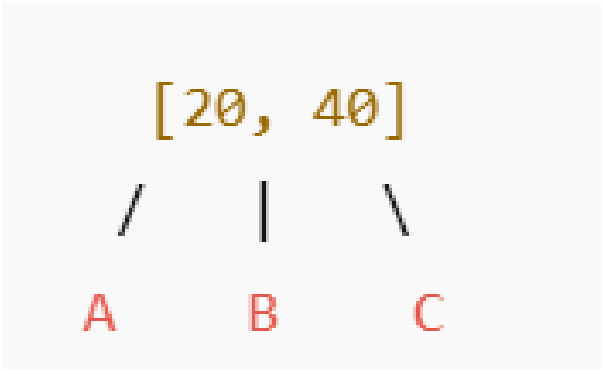
- Each node can hold a maximum of **2 keys (m-1)**.
- The keys in the subtrees are split according to the parent keys:
- Consider 24 35
- Left branch (i.e., 3 and 10) is less than 24.
- Middle branch (i.e., 31) is in between 24 35.
- Right branch (i.e. 50) is higher than 35.

What is a **B-tree**? Cont..

- They reduce the height of the tree, keeping operations fast.
- Useful for **disk-based storage** where reading a block is costly.
- Widely used in **databases, filesystems (e.g., NTFS(**New Technology File System)**, HFS+), and indexes**.

B-tree Properties

- Consider the following tree



- Two keys : 20, 40
- Children: 3 subtrees (A, B, C)

Degree (t): Minimum number of children per internal node. Controls are :Min/Max keys and children

Order (m): Maximum number of children per node. Controls are Max keys = m−1

m=2t

or

t=m/2

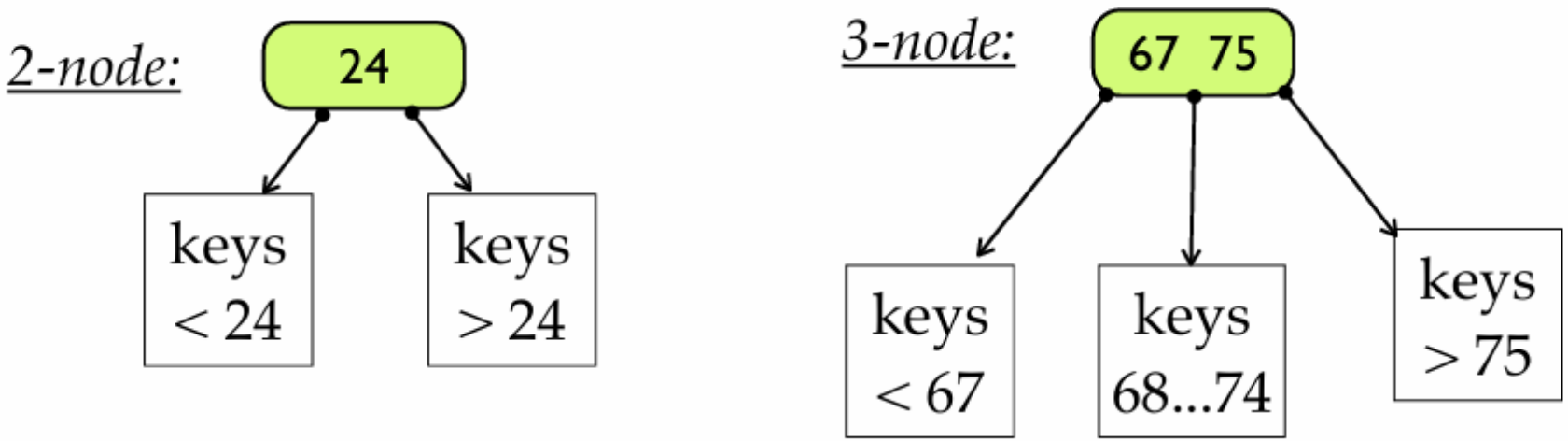
B-tree Properties

- **Degree** (t)(or **Minimum Degree**, denoted t):
- The minimum number of **children** a non-root internal node must have.
- Min keys = t−1
- Max keys = 2t−1
- Max children = 2t

- **Order(m)** (often denoted as m):**maximum number of children** a node can have.
- Max children = m
- Max keys = m−1

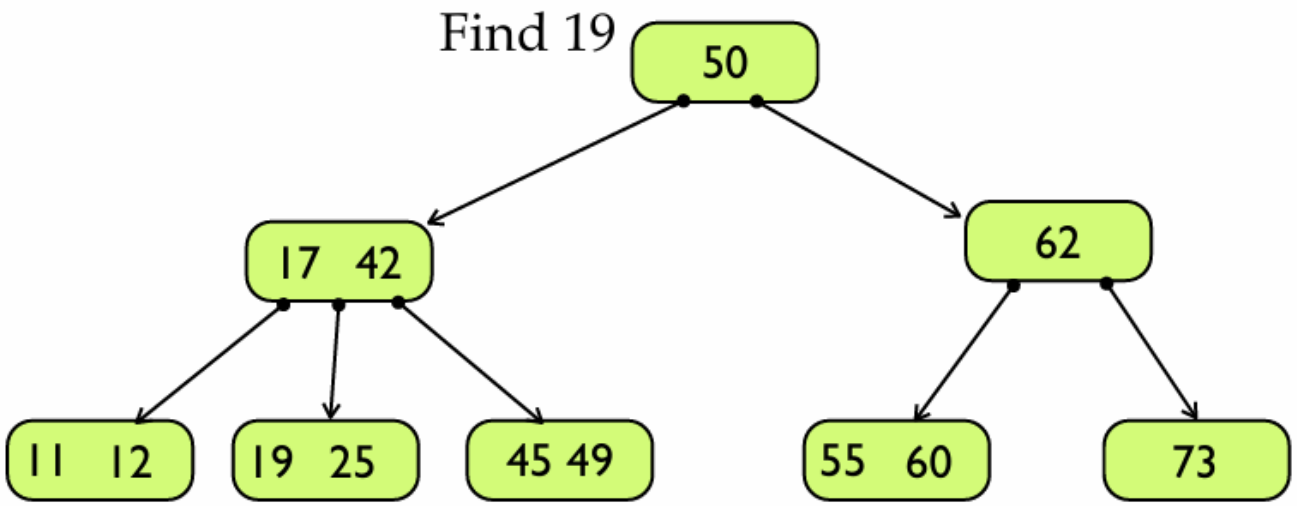
2,3 Trees

- All leaves are at the same level.
- Each internal node has either 2 or 3 children.
- If it has:
  - 2 children => it has 1 key
  - 3 children => it has 2 keys



Searching

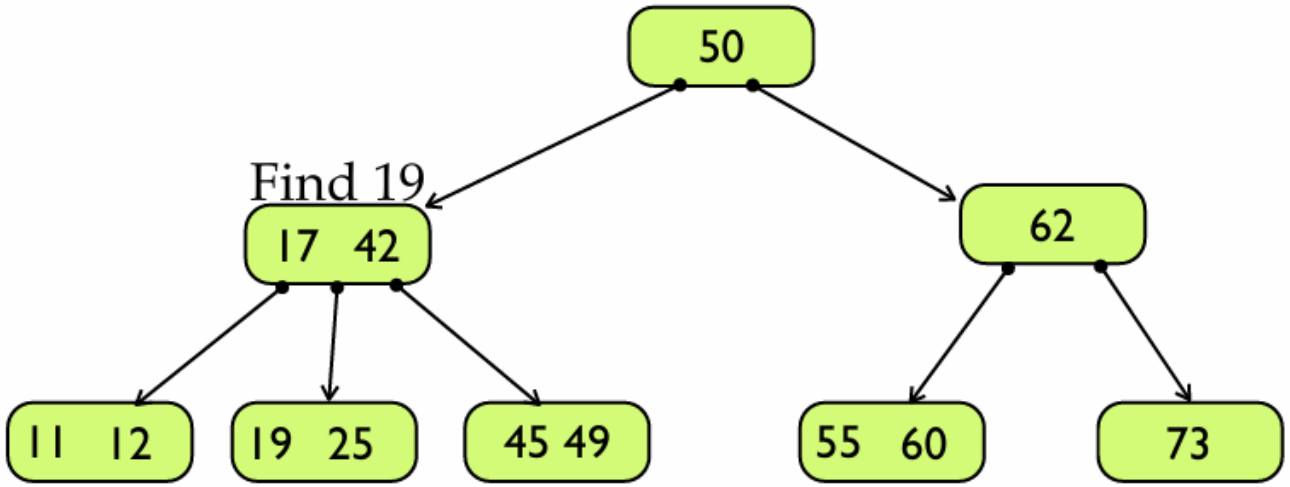
2,3 Tree Find (multiway searching)



Standard BST-type walk down the tree.
At each node have to examine each key stored there.

Searching

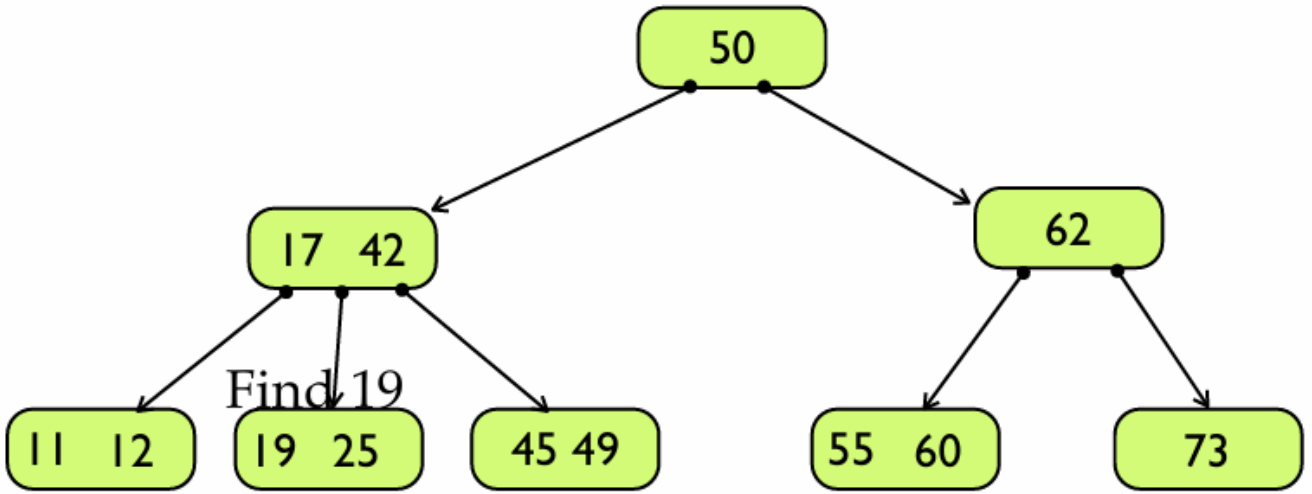
2,3 Tree Find (multiway searching)



Standard BST-type walk down the tree.
At each node have to examine each key stored there.

Searching

2,3 Tree Find (multiway searching)



Standard BST-type walk down the tree.
At each node have to examine each key stored there.

How to build a B-tree

In general:

•A **B-tree of minimum degree (t)** allows:

- Up to **2t−1** keys per node.
- Up to **2t** children per node.

- **Minimum degree** $t = 2$

  (Sometimes called order 4.)
- **Per-node limits (except the root):**
  - Keys per node: **min** = $t - 1 = 1$, **max** = $2t - 1 = 3$
  - Children per node: **min** = $t = 2$, **max** = $2t = 4$
- **Root** may have 1–3 keys and 0–4 children.

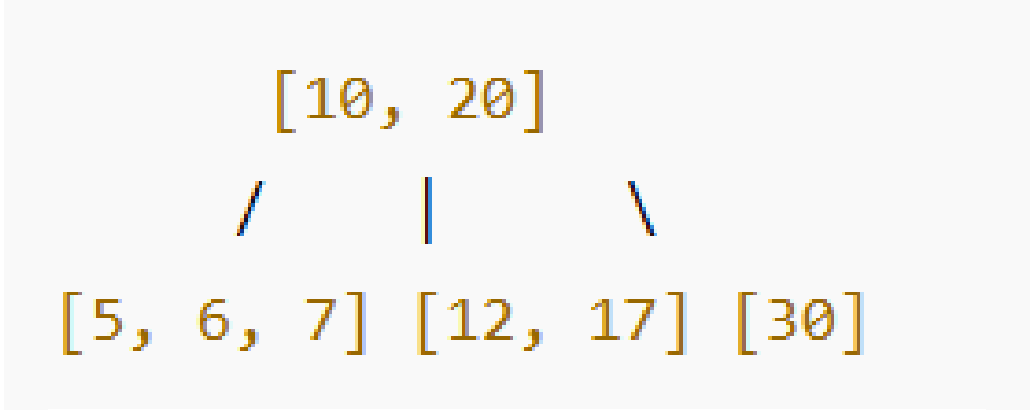
We'll **insert** the numbers in this order:

Minimum Degree t	Max Keys per Node	Max Children per Node	Often Called
2	3	4	2-3-4 Tree or 3-4 Tree
3	5	6	4-5 Tree (less common term)

Example: B-tree **of order 3 (also called a 2-3 tree)**:

- **B-tree of Order 3:**
- Each node can have:
  - **1 or 2 keys**
  - **2 or 3 children**
- Keys in nodes are **sorted**
- All leaf nodes are at the **same depth**

- E.g. Insert the following values in order :10, 20, 5, 6, 12, 30, 7, 17



Root has two keys: `10` and `20`

It splits the key space into three ranges:

- Left child has keys `< 10` → `[5, 6, 7]`
- Middle child has keys `10 ≤ key < 20` → `[12, 17]`
- Right child has keys `≥ 20` → `[30]`

All **leaf nodes** are at the same level → balanced.

Insertion rules

- **Search** down to the appropriate leaf.
- **If the leaf has < 3 keys**, insert in sorted order—done.
- **If the leaf is full (3 keys)**, **split**:
  - Median key **moves up** to the parent.
  - Left and right halves become two nodes.
  - If the parent is full, split the parent too (this can cascade to the root, possibly creating a new root).

Algorithm -Insertion Operation

- If the tree is empty, allocate a root node and insert the key.
- Update the allowed number of keys in the node.
- Search the appropriate node for insertion.
- If the node is full, follow the steps below.
- Insert the elements in increasing order.
- Now, there are elements greater than its limit. So, split at the median.
- Push the median key upwards and make the left keys as a left child and the right keys as a right child.
- If the node is not full, follow the steps below.
- Insert the node in increasing order.

Question 1

- Insert the following numbers into an initially empty B-tree to build a B-tree of order(m) =4
- [5,3,21,9,13,22,7,10,11,14,8,16]

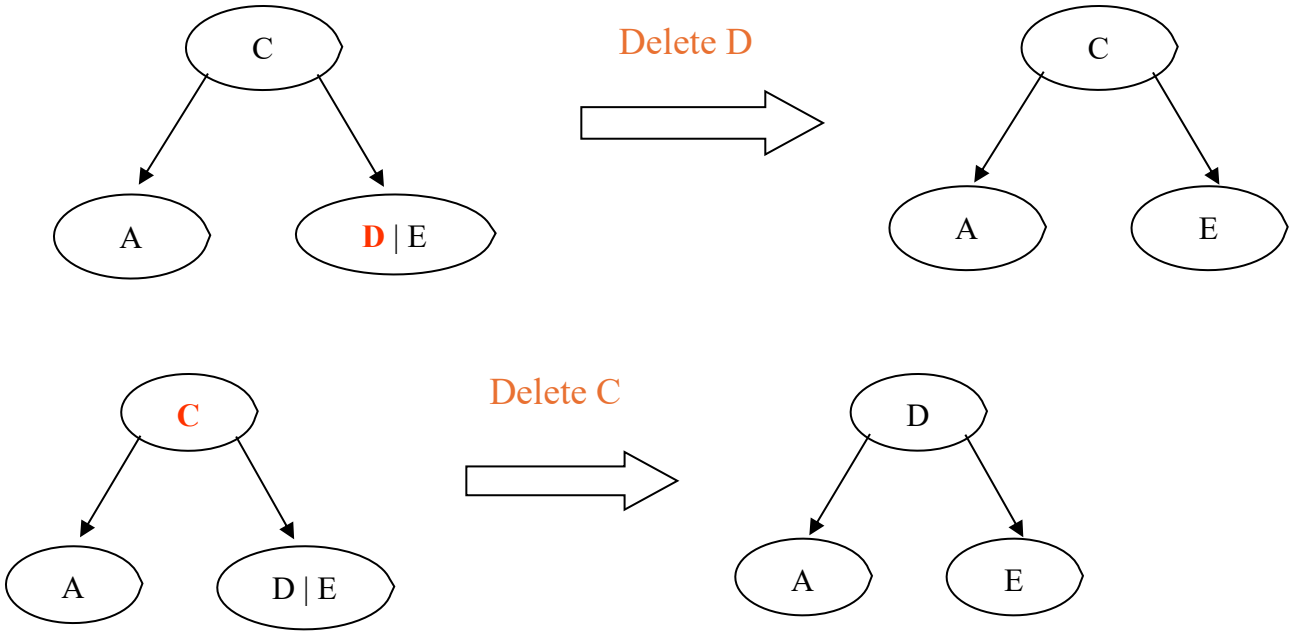
Deletion

- If the entry to be deleted is not in a leaf, swap it with its successor (or predecessor) under the natural order of the keys. Then delete the entry from the leaf.
- If leaf contains more than the minimum number of entries, then one can be deleted with no further action.

Deletion operation

- **Case 1** − If the key to be deleted is in a leaf node and the deletion does not violate the minimum key property, just delete the node.
- **Case 2** − If the key to be deleted is in a leaf node but the deletion violates the minimum key property, borrow a key from either its left sibling or right sibling. In case if both siblings have exact minimum number of keys, merge the node in either of them.
- **Case 3** − If the key to be deleted is in an internal node, it is replaced by a key in either left child or right child based on which child has more keys. But if both child nodes have minimum number of keys, there merged together.
- **Case 4** − If the key to be deleted is in an internal node violating the minimum keys property, and both its children and sibling have minimum number of keys, merge the children. Then merge its sibling with its parent.

Deletion Example 1



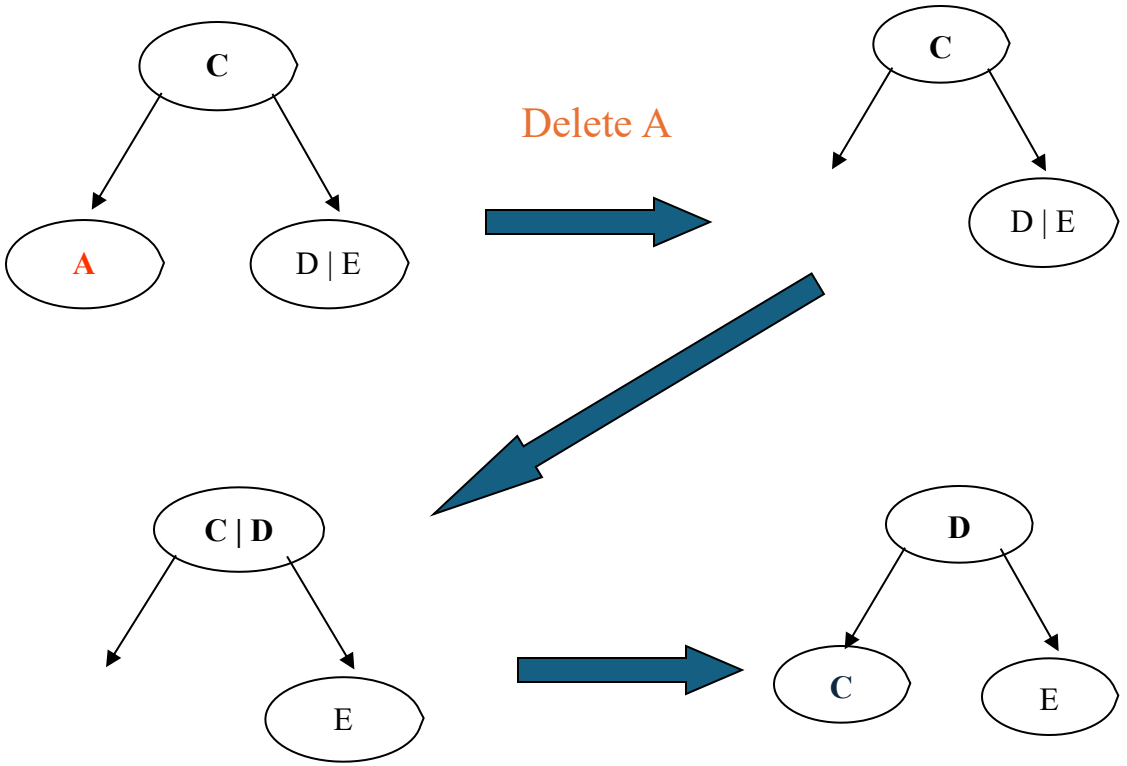
Successor is promoted, Element D

C is Deleted.

Deletion Cont...

- If the node contains the minimum number of entries, consider the two immediate siblings of the parent node:
- If one of these siblings has more than the minimum number of entries, then redistribute one entry from this sibling to the parent node, and one entry from the parent to the deficient node.
  - This is a rotation which balances the nodes
  - Note: all nodes must comply with minimum entry restriction.

Deletion Example 2

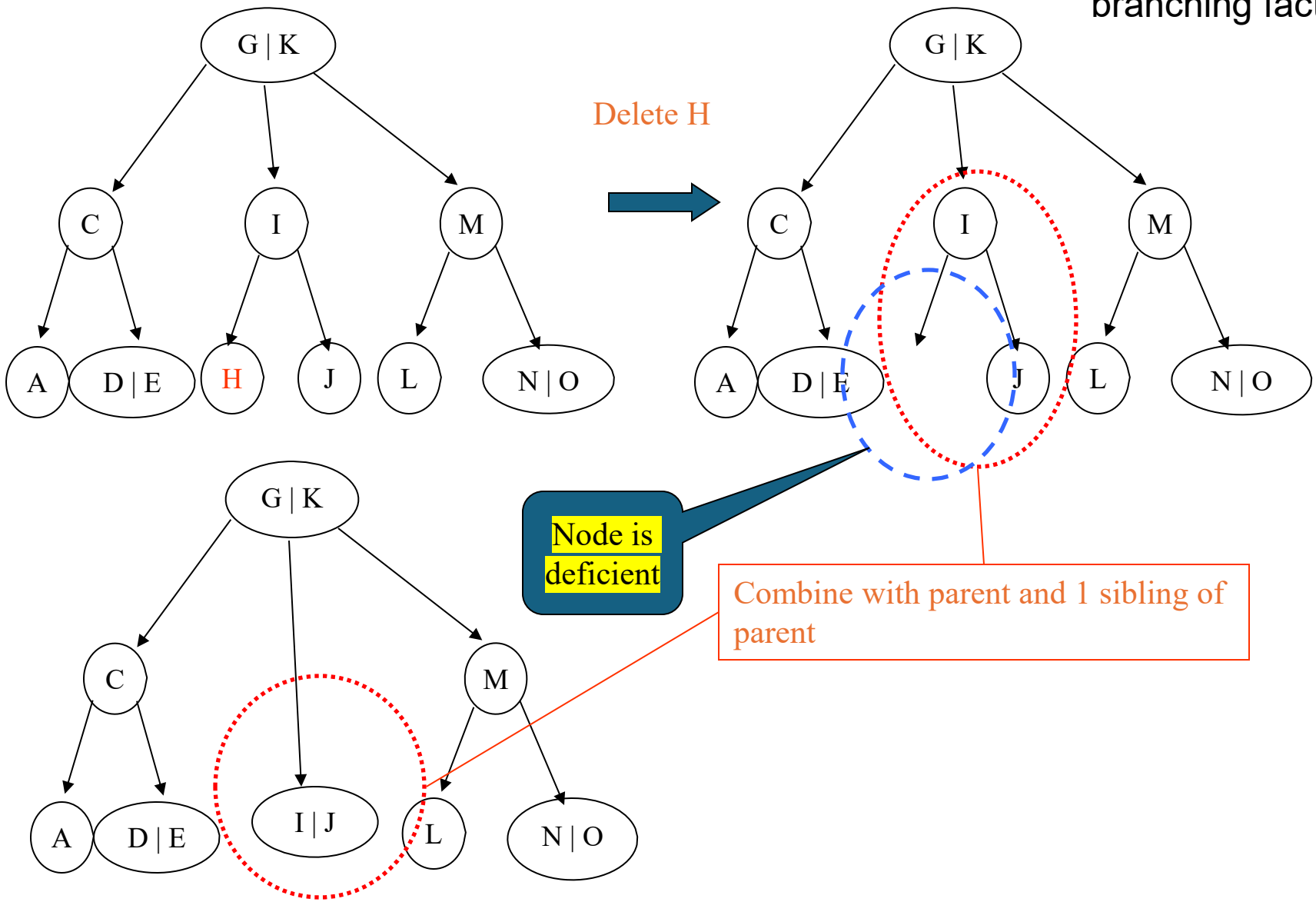


Deletion Cont...

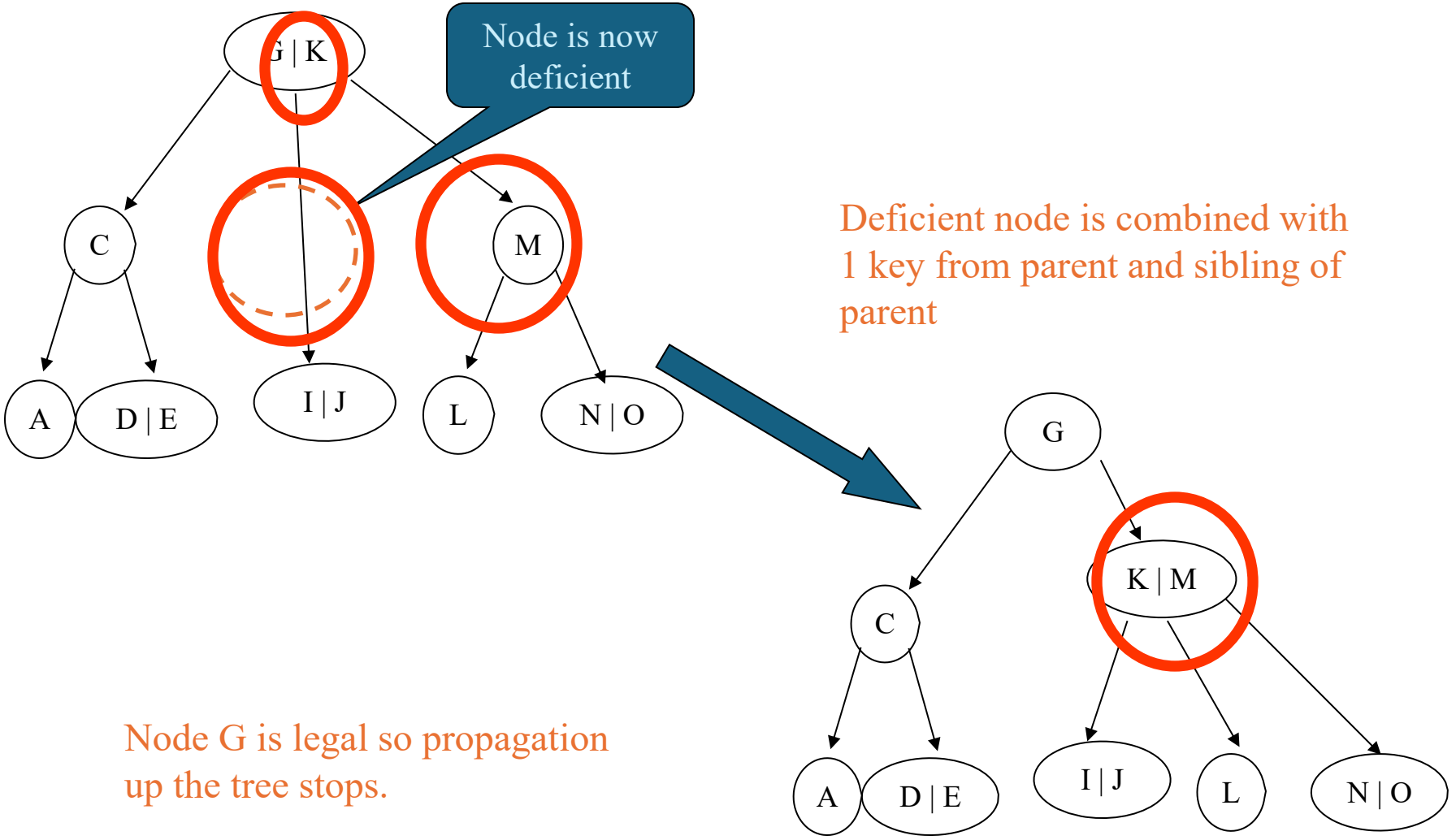
- If both immediate siblings have exactly the minimum number of entries, then merge the deficient node with one of the immediate sibling node and one entry from the parent node.
- If this leaves the parent node with too few entries, then the process is propagated upward.

Deletion Example 3

A node is **deficient** if it has **fewer than [t - 1] keys**, where t is the **minimum degree** (also called the branching factor or order).

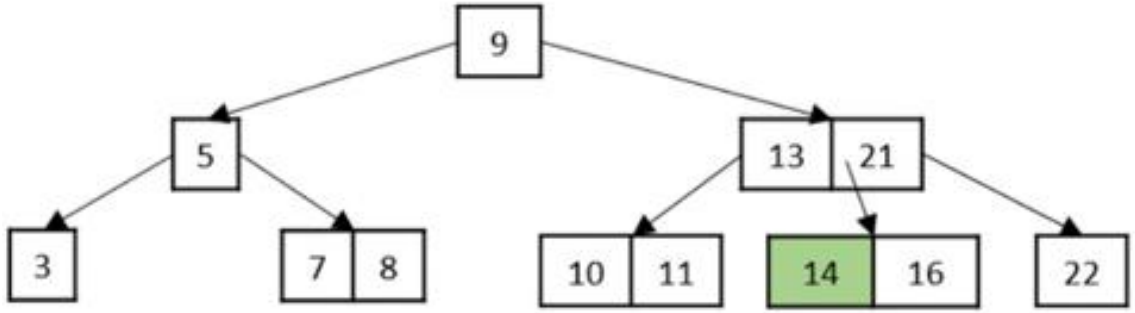


Deletion Example 3 Cont..



Question 2

- Consider the following Tree



Perform the following operations:

1) Delete node 14
2) Delete node 3
3) Delete node 13
4) Delete node 5

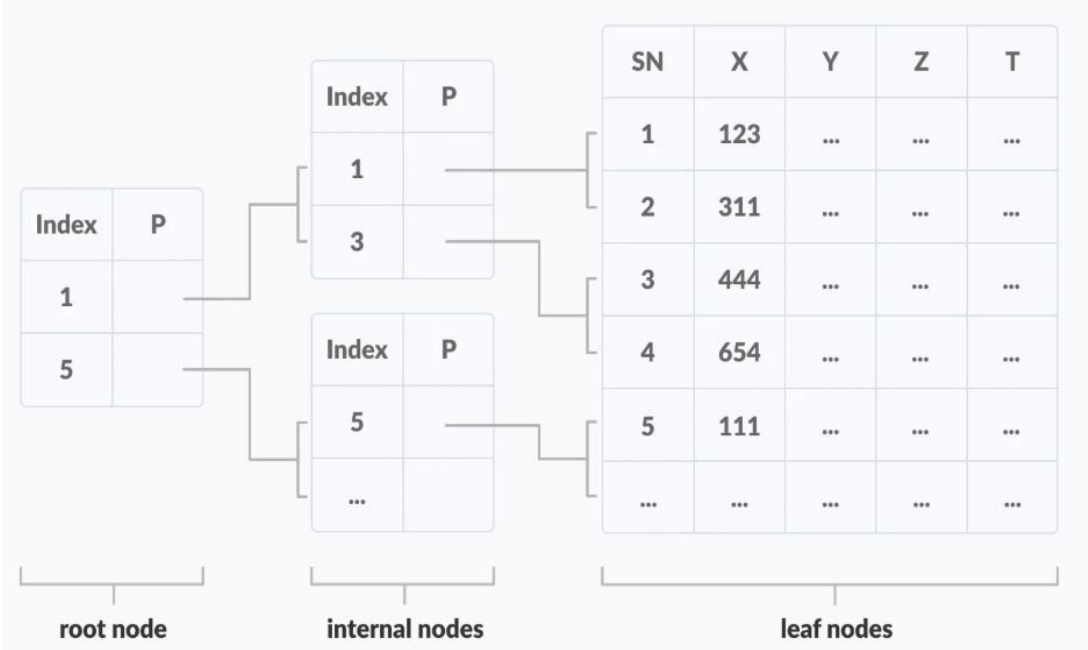
- **Note:** All deletions should be performed on the original tree.

Question 3

Construct a B-tree of order 5 using the following numbers:
1, 12, 8, 2, 25, 6, 14, 28, 17, 7, 52, 16, 48, 68, 3, 26, 29, 53, 55, 45

B+ Tree

- A B+ tree is an advanced form of a self-balancing tree in which all the values are present in the leaf level.
- An important concept to be understood before learning B+ tree is multilevel indexing.
- In multilevel indexing, the index of indices is created as in figure below. It makes accessing the data easier and faster.



Multilevel Indexing using B+ tree

What is a B+ Tree?

- A **B+ Tree** is an **advanced multi-way search tree** commonly used in **databases and file systems** to store and manage **large blocks of sorted data** efficiently.
- It is an extension of the **B-tree** with one major difference: All **data records (actual values) are stored only in the leaf nodes**, and **internal nodes only store keys used for indexing**.

Key Differences Between B-tree and B+ Tree

Feature	**B-tree**	**B+ Tree**
Data Storage	Keys and data are stored in both internal and leaf nodes.	All data is stored **only in the leaf nodes**. Internal nodes store only keys.
Leaf Nodes	Not necessarily linked.	All leaf nodes are linked together (linked list).
Search Efficiency	May require traversal to leaf or internal nodes.	Always reaches leaf level → **more predictable and efficient**.
Range Queries	Less efficient due to lack of linked leaves.	**Efficient** using linked leaves (sequential access).
Internal Node Size	Stores both keys and values (data).	Stores only keys (no data), allowing more keys per node.
Access Time	Slower for range queries.	Faster for both range and full table scan operations.
Use Case	Suitable for general in-memory storage.	Preferred for disk-based storage like databases and file systems.

Key Characteristics of B+ Trees:

Feature	Description
Multi-way Search Tree	Each internal node can have multiple children (based on order m).
Index in Internal Nodes	Internal nodes contain only keys (no data).
Data in Leaf Nodes	All actual data entries (records) are in the **leaf nodes**.
Linked Leaf Nodes	Leaf nodes are linked in a **linked list** for fast sequential access.
Balanced	Like B-trees, B+ trees are **height-balanced**. All leaves are at the same level.

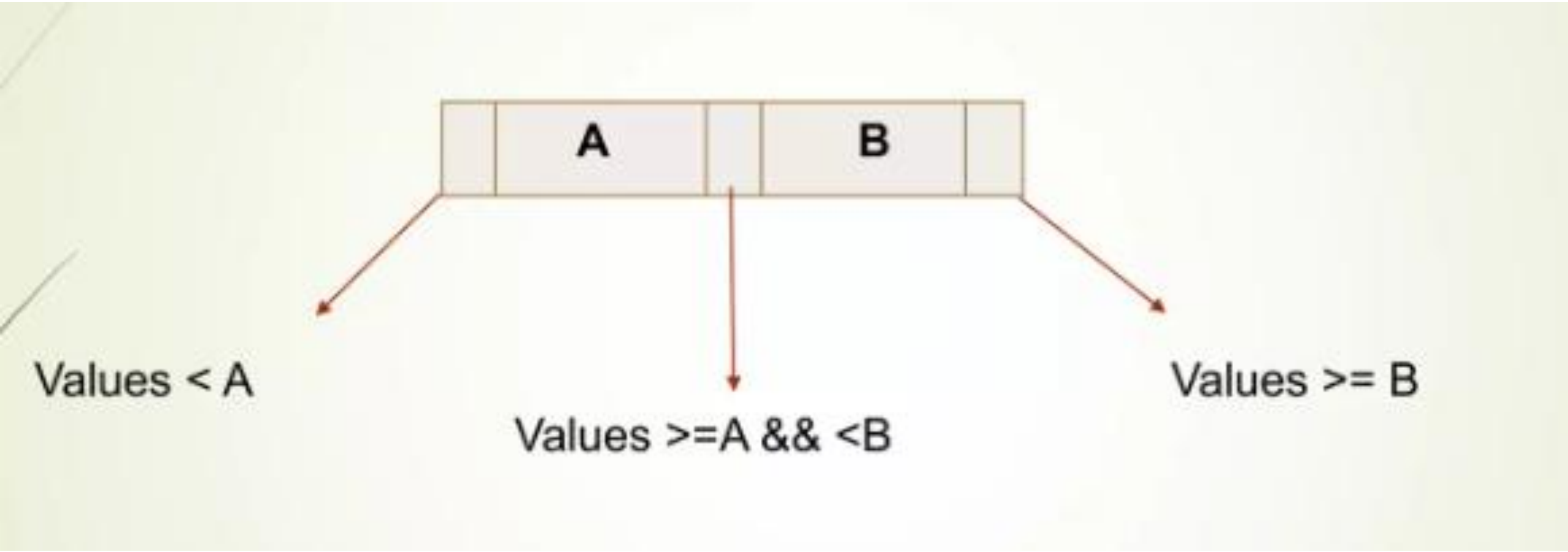
Applications:

- **Database indexing systems** (e.g., MySQL, PostgreSQL)
- **File systems** (e.g., NTFS, ReiserFS)
- **Key-value stores and NoSQL DBs**

B^+ Trees

- Every node has pointers to other nodes and key values.
- If there are **N** pointers then there can be **N-1** number of key values.
- Key values are repeated from root node to leaf node because data pointers are located at leaf nodes.

B^+ Tree Structure



B^+ tree Insertion

- Lets take the following elements
  2, 5, 7, 10, 13, 16, 20, 22, 23, 24….
- Let be number of pointer, indirectly, **3** key values can be there in a node at a maximum case….
- Insert 2,5,7….

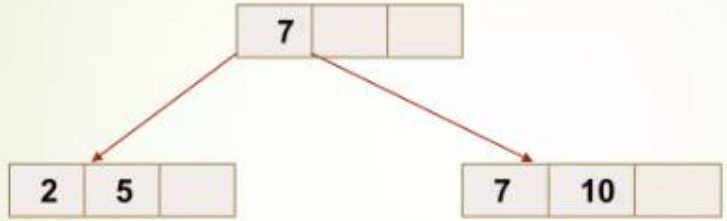
- Now insert 10….

B^+ tree Insertion (Cont....)

- When we insert 10 then node will be break down into two..
- After inserting 10..

- There should be a parent to look after when there nodes > 1, so copy 7 into parent node....

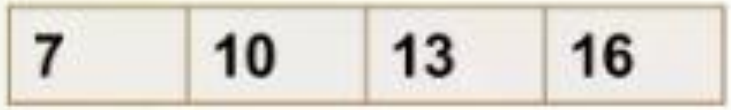
- Lets see....



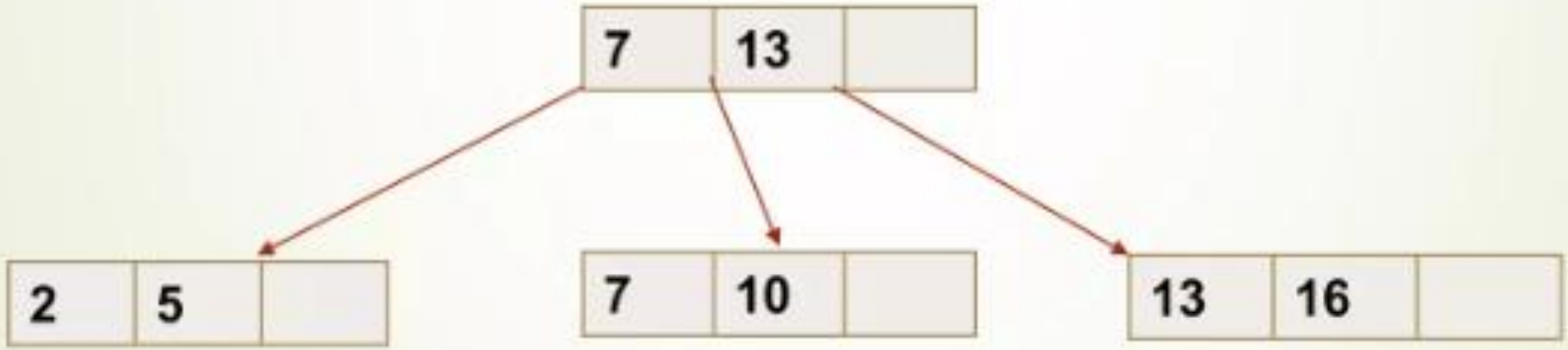
- Insert 13, 16..... Node again breaks

B^+ tree Insertion (Cont....)

- After inserting 13 and 16....
- Nodes over flows....



- After insertion....



Question 4

- Consider the following set of numbers:
  10, 20, 5, 6, 12, 30, 7, 17
- **a)** Create a B+ Tree using the above dataset.
  Note: You may present the step-by-step procedure of B+ Tree construction, including explanations at each intermediate step.
- **b)** Why are the leaf nodes linked in a B+ Tree?